Smart Motor Health Diagnostics: A Real-Time Monitoring System with IMU, IR Sensor, and Free-RTOS

Group: Project Group 2
Sofia Makowska, Jeje Dennis,
Aniket Fasate, Madison O'Neil

Central Idea

- Problem:
  - Early-stage DC motor faults often go undetected with conventional monitoring
  - Small issues (imbalance, bearing wear, misalignment) can escalate into major failures which causes costly repairs, downtime, and safety risks

- Solution:
  - Utilize IMU sensors, IR sensors, and a real time operating system (FreeRTOS) to continuously monitor the motor vibrations and rotational speed
  - Use machine learning to accurately track the remaining useful life (RUL) of the motor for predictive maintenance

Objectives and Scope:

Level 1

Nov.3 - Nov.17

Goal: IMU sensor working and accurately tracking the motor's motion

Level 2

Nov.17 - Dec. 8

Goal: evolving into a diagnostic system capable of detecting and identifying faults while suggesting potential root causes

Level 3

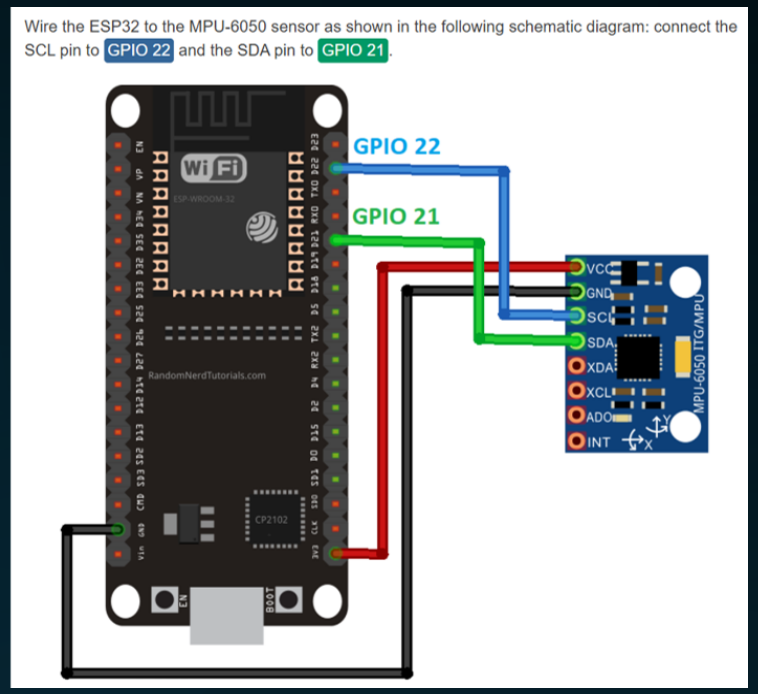
Dec. 8 – Dec. 14

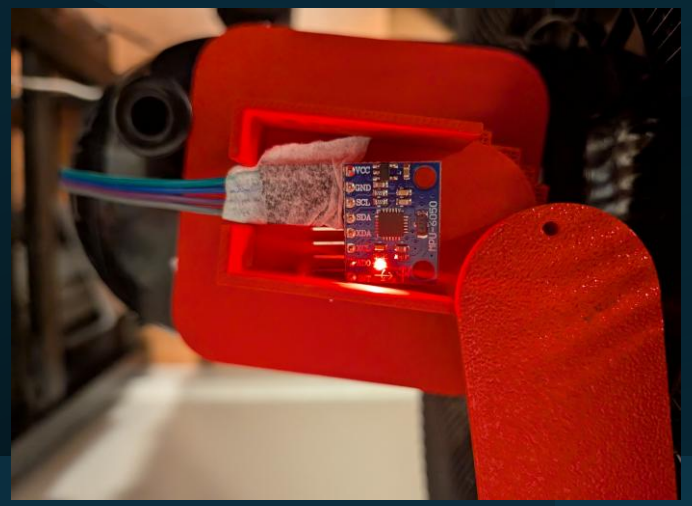
Goal: achieving full sensor fusion by integrating both the IMU and the IR sensor for synchronized data collection

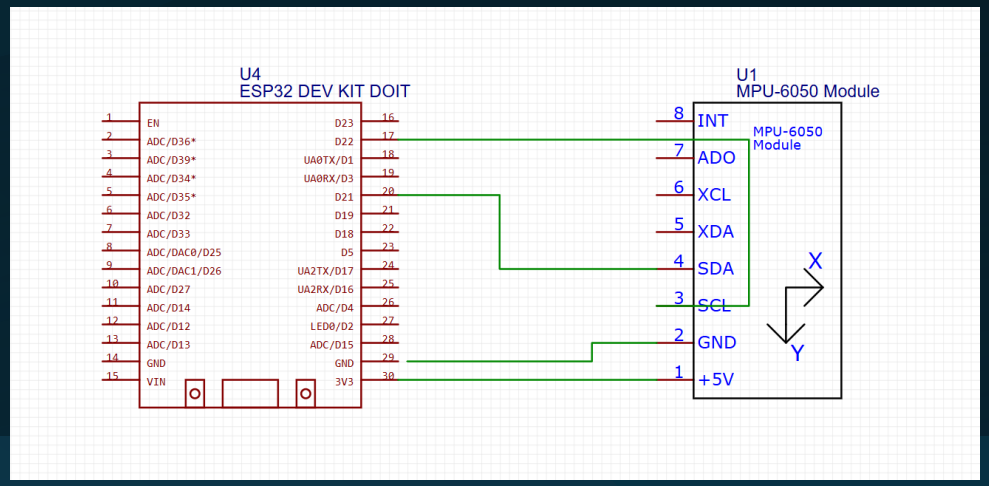
Current Progress: Experimental Setup and Collecting raw IMU data

Connected the ESP32 with the MPU-6050 and processed sensor data through Arduino/ESP32 code

Mounted motor system on top of a fan to collect more measurable vibration data

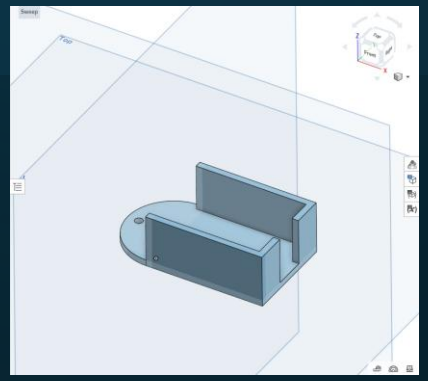
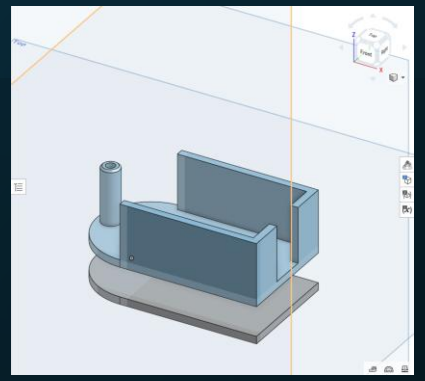
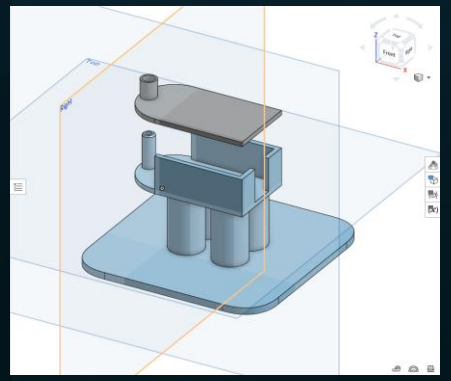
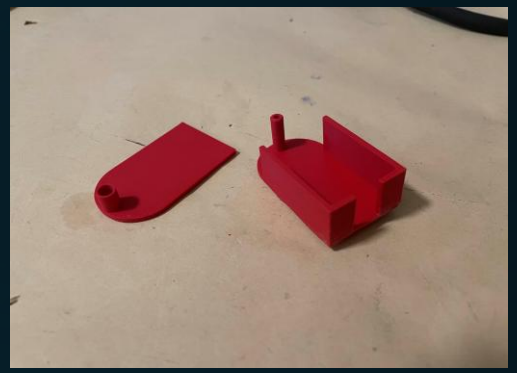






Approach: Creating the Test Bench Housing

- Several iterations for ergonomics and fit

	Model 1:	Model 2:	Model 3:
Digital			
Physical			

Approach: Data acquisition and processing are orchestrated using the Free-RTOS kernel to enable reliable real-time operation.

Core 0: Executes a high-priority sensor acquisition task that reads MPU6050 accelerometer (Ax, Ay, Az), gyroscope (Gx, Gy, Gz), and temperature data at 100 Hz using the I2C interface, then sends each sample to a 2048-depth queue in a non-blocking, deterministic manner to ensure zero-jitter timing

Core 1: Executes a lower-priority processing and communication task that dequeues the sensor samples from Core 0, applies a 40-sample moving average filter to reduce high-frequency noise while preserving the underlying signal characteristics, formats both raw and filtered data as CSV, and transmits the results over the configured serial port

156,000+ samples collected across 4 fault types

Healthy Data	Imbalance	Misalignment	Bearing Fault
34,592 Samples	37, 922 Samples	33,092 Samples	19,347 Samples

Motor Health Monitoring Using Multi-Domain Feature Extraction and Random Forests

Raw CSV Data
(6 files, 156K samples)

→ **Sliding Window Feature Extract**
(1000 Samples, 50% overlap)

→ **372 Samples 144 Features**

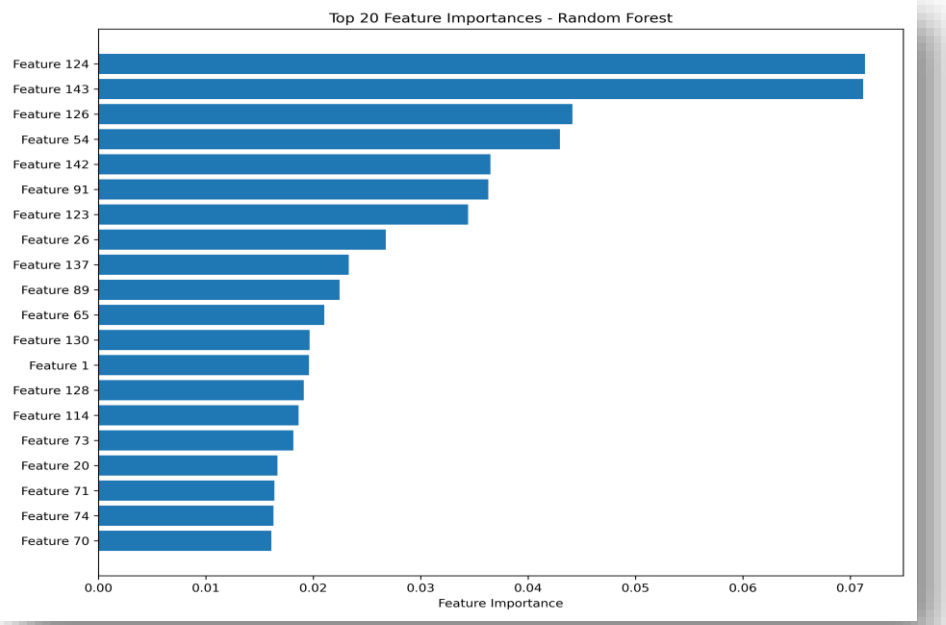
→ **Train/Test Split**
297 train, 75 test (80/20)

→ **ML Model training**
Random Forest

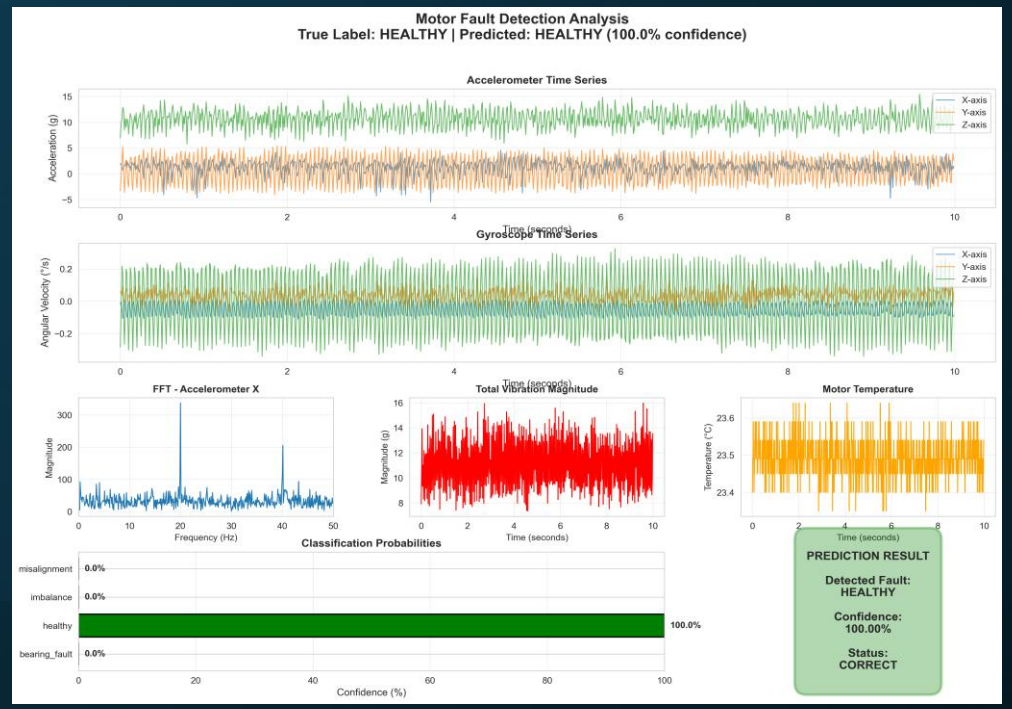
→ **More than 94% accuracy**

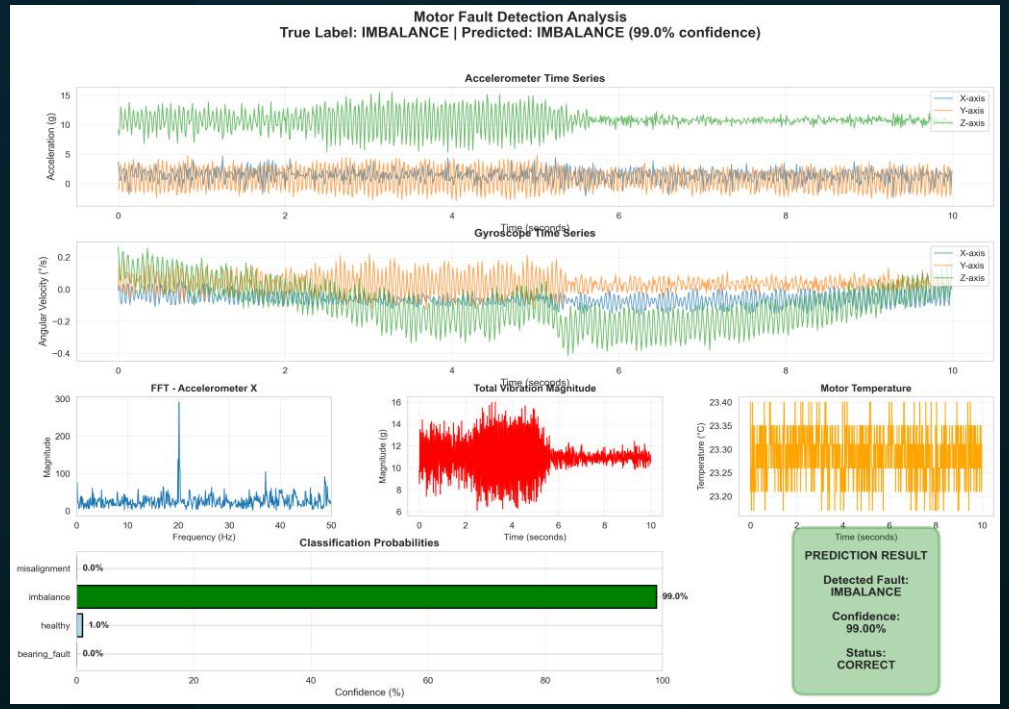
372 Samples, 144 Features

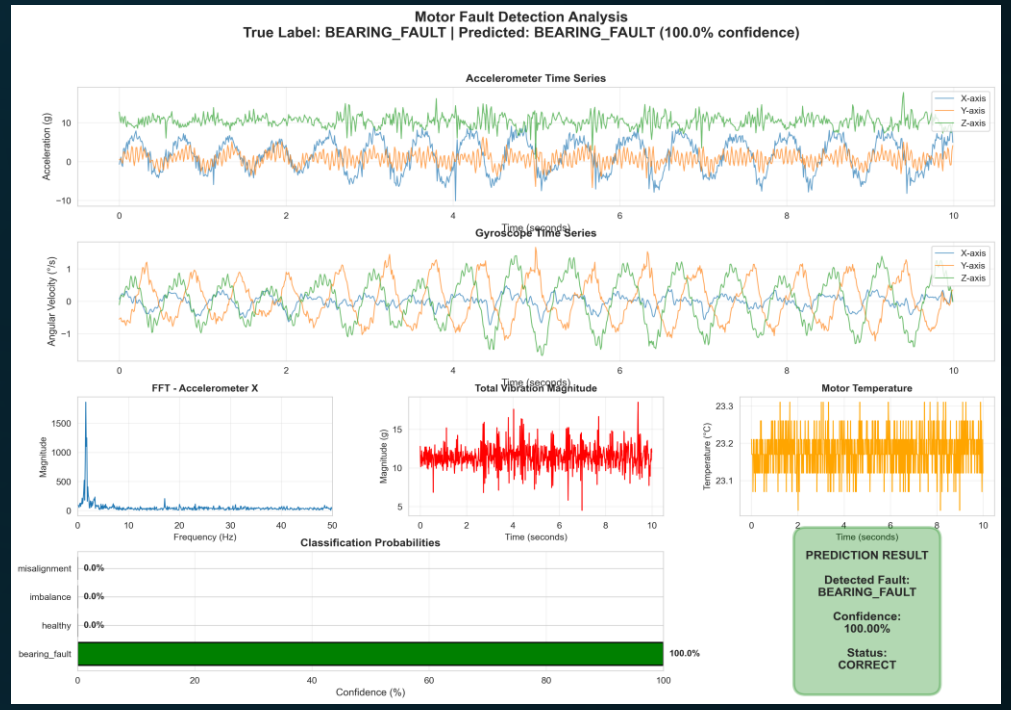
- **Time-Domian Features(54)**
  Mean, Std, Variance, Min, Max, Range, RMS...
- **Frequency Domain Features(72)**
  FFT peaks, spectral centroid, Mean, Std, Max of FFT magnitudes
- **Vibration Metrics(6)**
  Combined acceleration magnitude, Combined gyroscope magnitude, Mean, Std
- **Temperature Features(3)**
  Mean, Std, Max Temperature
- **Advanced Metrics(9)**
  Spectral properties

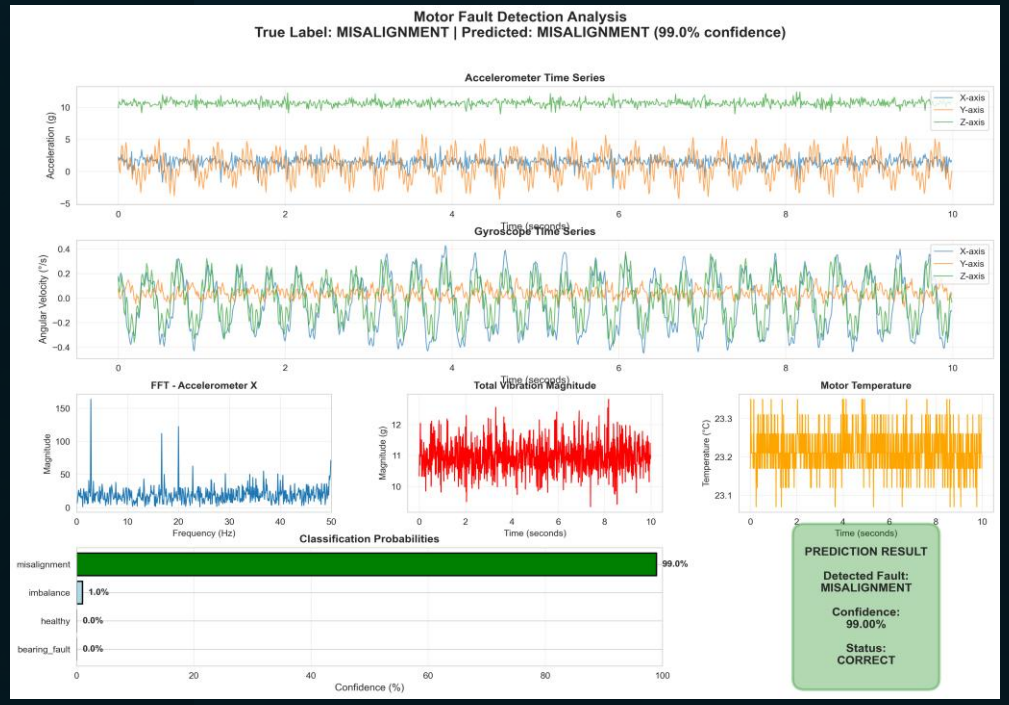


Results: Final Testing of Motor Diagnostic





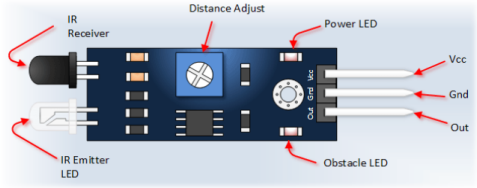




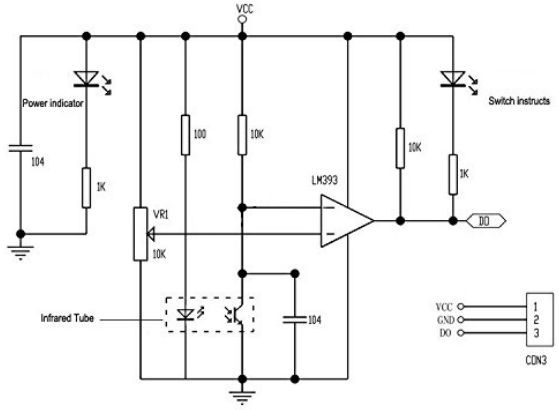
Next Steps:

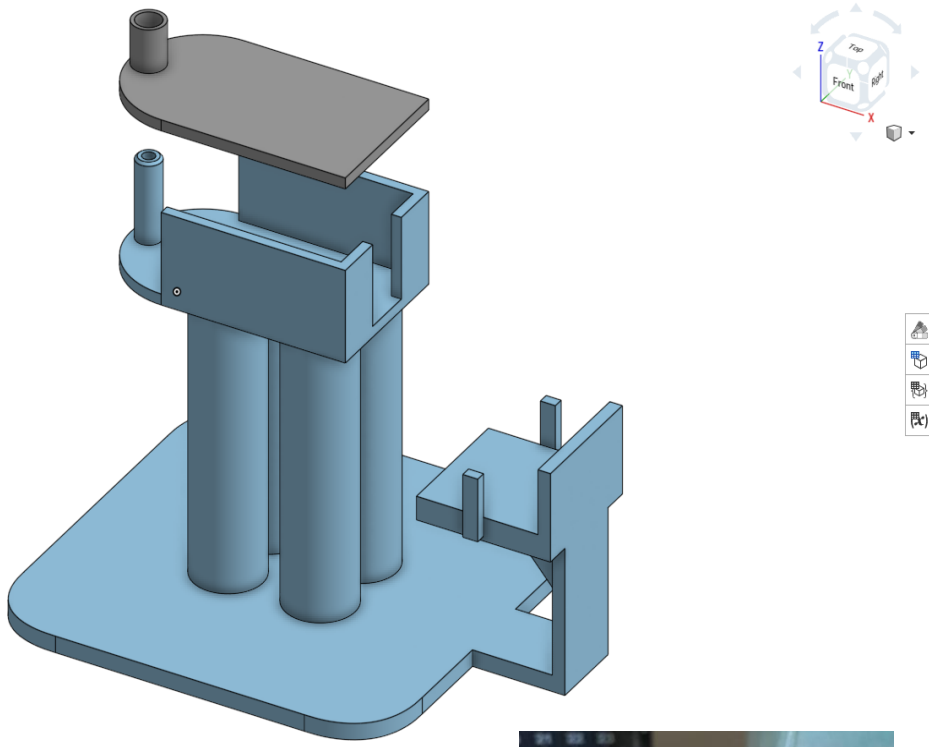
Adding Fan and IR sensor components

- Updated test bench housing with elevation and clearance for fan as well as a secondary chassis for holding the IR sensor



Pin, Control Indicator	Description
Vcc	3.3 to 5 Vdc Supply Input
Gnd	Ground Input
Out	Output that goes low when obstacle is in range
Power LED	Illuminates when power is applied
Obstacle LED	Illuminates when obstacle is detected
Distance Adjust	Adjust detection distance. CCW decreases distance. CW increases distance.
IR Emitter	Infrared emitter LED
IR Receiver	Infrared receiver that receives signal transmitted by Infrared emitter.





Learning Curve

several practical challenges

1

Multiple 3D-printed sensor mount

2

Small DC motor → Standing fan motor

3

Replicating Fault Conditions

Thank you!

Any Questions?

Github: https://github.com/AniketFasate27/EECE5554-Final_Project/tree/main